STOCKFLOW

Inventory Management System — B2B SaaS

Case Study Solution

Candidate: Vaishnavi Sadashiv Raut	**Date:** April 2026
Role: Software Engineering Intern	**Platform:** StockFlow B2B SaaS

This document contains full solutions to all 3 parts of the take-home case study.

Part 1: Code Review & Debugging

1.1 — Issues Identified in the Original Code

Issue #	Problem Found	Category	Production Impact	Severity
#1	No input validation / missing field checks	Security & Robustness	KeyError crashes server if any field is absent; malformed JSON goes undetected	CRITICAL
#2	No error handling / no try-except block	Robustness	Any DB exception surfaces as an unhandled 500 with stack traces exposed to client	CRITICAL
#3	Two separate db.session.commit() calls — no atomicity	Data Integrity	If second commit fails, product row exists with NO inventory record — silent data corruption	CRITICAL
#4	No SKU uniqueness check before insert	Business Logic	Duplicate SKUs silently overwrite or cause DB-level IntegrityError with no user-friendly message	HIGH
#5	Price not validated as positive decimal	Business Logic	Negative or zero prices accepted, distorting reports and financials	HIGH
#6	No authentication / authorization check	Security	Any unauthenticated user can add products to any warehouse — complete access control bypass	CRITICAL
#7	warehouse_id not verified to exist / belong to company	Business Logic	Products can be created in warehouses owned by other companies (multi-tenant data leak)	CRITICAL
#8	initial_quantity not validated (negative values allowed)	Business Logic	Negative quantities break stock calculations and alerts	HIGH
#9	No HTTP status codes in response	API Design	All responses return 200 even for errors, breaking client error handling	MEDIUM
#10	No logging	Observability	Production incidents are impossible to diagnose without any audit trail	MEDIUM
#11	price stored without decimal precision enforcement	Data Integrity	Floating-point price inaccuracies on financial calculations / invoices	MEDIUM

1.2 — Detailed Explanation of Each Issue

Issue #1 & #2: No Validation + No Error Handling

The endpoint blindly accesses data['name'], data['sku'], etc. If any key is missing — or if request.json returns None (non-JSON Content-Type) — Python raises a KeyError or TypeError, crashing with a 500 error. More critically, no try/except wraps the database operations, so any DB constraint violation propagates as an ugly error to the client.

Issue #3: Non-Atomic Double Commit (Most Dangerous Bug)

This is the most severe data integrity bug. Between the two commit() calls, the application is in an inconsistent state. If the server crashes, a network glitch occurs, or the Inventory insert fails for any reason, the database is left with an orphaned Product record that has no corresponding inventory entry. This causes:

- Products appearing in catalog with no stock (visible to customers)
- Inventory queries returning NULL or crashing due to missing JOIN rows
- Supplier reorder alerts being skipped for 'ghost' products

Issue #4: Missing SKU Uniqueness Enforcement in Application Layer

While the DB may have a UNIQUE constraint on sku, the application code doesn't proactively check this. The result is a raw IntegrityError thrown to the client — not a user-friendly 409 Conflict. The application should query for existing SKUs before attempting insert and return a meaningful error.

Issues #6 & #7: Authentication & Multi-Tenant Security Holes

In a B2B SaaS, every company's data must be isolated. This endpoint has no @login_required decorator and no validation that the authenticated user's company owns warehouse_id. A malicious actor could enumerate warehouse IDs and inject products into competitor inventories — a GDPR/compliance nightmare.

1.3 — Corrected Implementation

Language/Framework: Python 3.10+ with Flask & SQLAlchemy

Assumptions made about schema: Product has (id, name, sku, price, created_at); Inventory has (id, product_id, warehouse_id, quantity); Warehouse has (id, company_id).

```
from flask import request, jsonify, g
from decimal import Decimal, InvalidOperation
from sqlalchemy.exc import IntegrityError
import logging

logger = logging.getLogger(__name__)

@app.route('/api/products', methods=['POST'])
@login_required  # FIX #6: Enforce authentication
def create_product():
    # —— FIX #1: Validate Content-Type and parse body safely ——
```

```
    data = request.get_json(silent=True)
    if not data:
        return jsonify({'error': 'Invalid or missing JSON body'}), 400

    # ── FIX #1: Required field validation ──
    required_fields = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    missing = [f for f in required_fields if f not in data]
    if missing:
        return jsonify({'error': f'Missing required fields: {missing}'}), 400

    # ── FIX #5 & #8: Type + business rule validation ──
    try:
        price = Decimal(str(data['price']))
        if price <= 0:
            raise ValueError
    except (InvalidOperation, ValueError):
        return jsonify({'error': 'price must be a positive decimal'}), 400

    try:
        initial_qty = int(data['initial_quantity'])
        if initial_qty < 0:
            raise ValueError
    except (ValueError, TypeError):
        return jsonify({'error': 'initial_quantity must be a non-negative
integer'}), 400

    warehouse_id = data['warehouse_id']

    # ── FIX #7: Verify warehouse belongs to authenticated company ──
    warehouse = Warehouse.query.filter_by(
        id=warehouse_id,
        company_id=g.current_user.company_id
    ).first()
    if not warehouse:
        return jsonify({'error': 'Warehouse not found or access denied'}), 404

    # ── FIX #4: Proactive SKU uniqueness check ──
    if Product.query.filter_by(sku=data['sku']).first():
        return jsonify({'error': f"SKU '{data['sku']}' already exists"}), 409

    # ── FIX #3: Single atomic transaction ──
    try:
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=float(price),
            warehouse_id=warehouse_id
        )
        db.session.add(product)
        db.session.flush()  # Get product.id without committing

        inventory = Inventory(
            product_id=product.id,
            warehouse_id=warehouse_id,
            quantity=initial_qty
        )
        db.session.add(inventory)
        db.session.commit()  # ONE commit — fully atomic
```

```
        logger.info(  # FIX #10: Audit logging
            'Product created',
            extra={'user_id': g.current_user.id, 'sku': product.sku, 'product_id':
product.id}
        )

        # FIX #9: Return 201 Created
        return jsonify({
            'message': 'Product created successfully',
            'product_id': product.id,
            'sku': product.sku
        }), 201

    except IntegrityError as e:
        db.session.rollback()  # FIX #2: Always rollback on failure
        logger.error('DB integrity error creating product', exc_info=e)
        return jsonify({'error': 'Database constraint violation'}), 409

    except Exception as e:
        db.session.rollback()
        logger.error('Unexpected error creating product', exc_info=e)
        return jsonify({'error': 'Internal server error'}), 500
```

1.4 — Summary of All Fixes Applied

Fix	Code Change	Why It Matters
#1	request.get_json(silent=True) + field check	Prevents KeyError crash, validates JSON
#2	try/except + db.session.rollback()	Safe error recovery, no stack trace leaks
#3	flush() then single commit()	Atomicity — both rows or neither row
#4	SKU lookup before insert	User-friendly 409 instead of DB error
#5	Decimal validation + >0 check	Prevents negative or zero prices
#6	@login_required decorator	Blocks unauthenticated requests
#7	Warehouse.company_id check	Enforces multi-tenant data isolation
#8	int() cast + >=0 check	Prevents negative inventory quantities
#9	Return 201/400/404/409/500	Correct HTTP semantics for API clients
#10	logger.info / logger.error	Observability for production debugging
#11	Decimal() then float()	Avoids floating-point price distortion

Part 2: Database Design

2.1 — Full Database Schema (SQL DDL)

```sql
-- ============================================================
-- STOCKFLOW DATABASE SCHEMA
-- Design: 3NF, multi-tenant, audit-ready, scalable
-- ============================================================

-- 1. COMPANIES (top-level multi-tenant entity)
CREATE TABLE companies (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(255) NOT NULL,
    slug          VARCHAR(100) UNIQUE NOT NULL,  -- URL-safe identifier
    plan_tier     ENUM('starter','pro','enterprise') DEFAULT 'starter',
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- 2. USERS (employees of companies)
CREATE TABLE users (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id    BIGINT NOT NULL,
    email         VARCHAR(320) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role          ENUM('admin','manager','staff') DEFAULT 'staff',
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE,
    INDEX idx_users_company (company_id)
);

-- 3. WAREHOUSES (belong to a company, can be in different locations)
CREATE TABLE warehouses (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id    BIGINT NOT NULL,
    name          VARCHAR(255) NOT NULL,
    address_line1 VARCHAR(255),
    city          VARCHAR(100),
    state         VARCHAR(100),
    country       CHAR(2) NOT NULL DEFAULT 'IN',  -- ISO 3166-1 alpha-2
    postal_code   VARCHAR(20),
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE,
    INDEX idx_warehouses_company (company_id)
);

-- 4. SUPPLIERS
CREATE TABLE suppliers (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id    BIGINT NOT NULL,  -- Supplier is scoped to a company
    name          VARCHAR(255) NOT NULL,
    contact_name  VARCHAR(255),
    contact_email VARCHAR(320),
```

```
    contact_phone VARCHAR(30),
    lead_time_days INT DEFAULT 7,   -- Typical reorder lead time
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE,
    INDEX idx_suppliers_company (company_id)
);

-- 5. PRODUCT CATEGORIES
CREATE TABLE product_categories (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id    BIGINT NOT NULL,
    name          VARCHAR(100) NOT NULL,
    low_stock_threshold INT DEFAULT 10,  -- Per-category threshold for alerts
    FOREIGN KEY (company_id) REFERENCES companies(id),
    UNIQUE (company_id, name)
);

-- 6. PRODUCTS (catalog — NOT warehouse-specific)
CREATE TABLE products (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    company_id    BIGINT NOT NULL,
    category_id   BIGINT,
    supplier_id   BIGINT,
    name          VARCHAR(255) NOT NULL,
    sku           VARCHAR(100) NOT NULL,
    description   TEXT,
    unit_price    DECIMAL(12, 4) NOT NULL,  -- 4 decimal places for precision
    is_bundle     BOOLEAN DEFAULT FALSE,
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES product_categories(id) ON SET NULL,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(id) ON SET NULL,
    UNIQUE (company_id, sku),   -- SKU unique per company
    INDEX idx_products_company (company_id),
    INDEX idx_products_category (category_id),
    INDEX idx_products_supplier (supplier_id)
);

-- 7. BUNDLE COMPONENTS (self-referential for bundle products)
CREATE TABLE bundle_components (
    bundle_product_id  BIGINT NOT NULL,
    component_product_id BIGINT NOT NULL,
    quantity           INT NOT NULL DEFAULT 1,
    PRIMARY KEY (bundle_product_id, component_product_id),
    FOREIGN KEY (bundle_product_id) REFERENCES products(id) ON DELETE CASCADE,
    FOREIGN KEY (component_product_id) REFERENCES products(id) ON DELETE RESTRICT
);

-- 8. INVENTORY (product quantity per warehouse)
CREATE TABLE inventory (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    product_id    BIGINT NOT NULL,
    warehouse_id  BIGINT NOT NULL,
    quantity      INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    reorder_point INT DEFAULT 10,   -- Product+warehouse specific threshold
    reorder_qty   INT DEFAULT 50,   -- How much to reorder
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
    FOREIGN KEY (warehouse_id) REFERENCES warehouses(id) ON DELETE CASCADE,
    UNIQUE (product_id, warehouse_id),
    INDEX idx_inventory_warehouse (warehouse_id),
    INDEX idx_inventory_product (product_id)
);

-- 9. INVENTORY_LEDGER (immutable audit trail of every stock change)
CREATE TABLE inventory_ledger (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    product_id    BIGINT NOT NULL,
    warehouse_id  BIGINT NOT NULL,
    user_id       BIGINT,
    change_type
ENUM('sale','purchase','adjustment','transfer_in','transfer_out','return') NOT
NULL,
    quantity_delta INT NOT NULL,   -- Positive=in, Negative=out
    quantity_after INT NOT NULL,   -- Snapshot of stock after change
    reference_id  VARCHAR(100),   -- Order/PO/adjustment reference
    notes         TEXT,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (warehouse_id) REFERENCES warehouses(id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
    INDEX idx_ledger_product (product_id),
    INDEX idx_ledger_warehouse (warehouse_id),
    INDEX idx_ledger_created (created_at)  -- For time-range queries
);
```

2.2 — Entity Relationship Overview

From Entity	Relationship	To Entity	Notes
Company	1 → Many	Warehouses	Each company owns multiple physical sites
Company	1 → Many	Products	Product catalog is company-scoped
Company	1 → Many	Suppliers	Supplier relationships are per-company
Product	Many → Many	Warehouse (via Inventory)	Same product exists in multiple warehouses
Product	1 → Many	Inventory Ledger	Every stock movement logged immutably
Product	Self-referential	Bundle Components	Bundle products contain other products
Supplier	1 → Many	Products	Supplier provides multiple products
Category	1 → Many	Products	Categories define low_stock_threshold

2.3 — Design Decisions & Justifications

Decimal(12,4) for Price

Using DECIMAL instead of FLOAT avoids IEEE 754 floating-point errors on financial calculations. 4 decimal places supports currencies with sub-cent precision (e.g., cryptocurrency, commodity trading).

Inventory Ledger (Append-Only Audit Log)

Rather than a single updateable quantity column, the inventory_ledger table records every change with a quantity_delta and quantity_after snapshot. This provides:

- Full audit trail for compliance and dispute resolution
- Ability to reconstruct inventory state at any point in time
- Detection of discrepancies between expected and actual stock

SKU Unique Per Company (not globally)

In a multi-tenant SaaS, Company A and Company B can both use SKU 'WID-001' without conflict. The UNIQUE(company_id, sku) constraint enforces this correctly at the database level as a compound unique key.

Reorder Point Stored on Inventory Row

The reorder_point is stored per (product, warehouse) combination — not just per product. A product may have different low-stock thresholds in different warehouses (e.g., 100 at a main distribution center vs. 10 at a satellite store).

Soft Deletes (is_active flags)

Warehouses, products, and suppliers are never hard-deleted; they are deactivated. This preserves referential integrity with historical ledger entries and prevents orphaned records in audit trails.

2.4 — Gaps Identified & Questions for Product Team

The following requirements are incomplete. I would raise these before building:

#	Gap / Missing Requirement	My Question to Product Team
1	Multi-supplier per product	Can a single product be sourced from multiple suppliers? If yes, we need a product_suppliers junction table with preferred_supplier flag.

#	Gap / Missing Requirement	My Question to Product Team
2	Bundle stock calculation	When a bundle is sold, does the system auto-deduct component stock? Who triggers this — the API or a background job?
3	Inventory transfers between warehouses	Is warehouse-to-warehouse stock transfer a first-class operation? It requires atomic deduct+add with a TRANSFER_OUT/TRANSFER_IN ledger pair.
4	Low-stock threshold granularity	Is the threshold global per product, per category, or per (product + warehouse)? My schema supports all three — which does the business need?
5	Sales activity definition	The spec says 'only alert for products with recent sales activity'. What is 'recent'? Last 7 days? 30 days? Is this configurable per company?
6	Days-until-stockout calculation	What sales velocity should we use — simple average, weighted, or demand forecasting? Over what look-back window?
7	Access control model	Can a 'staff' user see stock across all warehouses? Or only their assigned warehouse? Role-based visibility affects query design significantly.
8	Currency support	Is price always in a single currency per company, or do we need multi-currency pricing with exchange rates?

Part 3: API Implementation — Low Stock Alerts

Time allocated: 35 minutes | Examines: query design, business logic, edge cases, API correctness

3.1 — Assumptions Documented

#	Assumption Made	Rationale
A1	Schema from Part 2 is used (inventory, inventory_ledger, products, etc.)	Self-consistent solution
A2	'Recent sales activity' = at least 1 sale in the last 30 days	Configurable — added as a query param
A3	Days-until-stockout = current_stock / avg_daily_sales over last 30 days	Simple, explainable formula; 0 if no recent sales
A4	Low-stock threshold = inventory.reorder_point (per warehouse)	Supports warehouse-specific thresholds from Part 2
A5	Authenticated user's company_id is verified against company_id path param	Security: prevents cross-company data access
A6	Inactive products and inactive warehouses are excluded from alerts	No noise from archived data
A7	If a product has multiple suppliers, the preferred/first supplier is returned	Simplified for MVP; noted as a gap

3.2 — Full API Implementation

Endpoint: GET /api/companies/{company_id}/alerts/low-stock

Optional query params: ?days=30 (look-back window) | ?warehouse_id=X (filter to one warehouse)

```
from flask import request, jsonify, g
from datetime import datetime, timedelta
from sqlalchemy import text
import logging

logger = logging.getLogger(__name__)

@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
@login_required
def get_low_stock_alerts(company_id):
    """
    Returns low-stock alerts for all active products in a company.
    Only includes products with recent sales activity.
    Calculates days_until_stockout based on sales velocity.
    """
```

```
    # ── Authorization: ensure user belongs to requested company ──
    if g.current_user.company_id != company_id:
        return jsonify({'error': 'Access denied'}), 403

    # ── Validate / parse query parameters ──
    try:
        days_lookback = int(request.args.get('days', 30))
        if not 1 <= days_lookback <= 365:
            raise ValueError
    except ValueError:
        return jsonify({'error': 'days must be an integer between 1 and 365'}),
400

    warehouse_filter_id = request.args.get('warehouse_id')
    if warehouse_filter_id:
        try:
            warehouse_filter_id = int(warehouse_filter_id)
        except ValueError:
            return jsonify({'error': 'warehouse_id must be an integer'}), 400

    cutoff_date = datetime.utcnow() - timedelta(days=days_lookback)

    try:
        # ── Core SQL Query ──
        # Strategy: JOIN inventory + products + warehouses + suppliers
        # Subquery computes avg daily sales from ledger (only 'sale' entries)
        # Filter: current_stock < reorder_point AND has_recent_sales

        sql = text('''
            WITH recent_sales AS (
                -- Aggregate sales per product+warehouse over the lookback window
                SELECT
                    il.product_id,
                    il.warehouse_id,
                    COUNT(*) AS sale_count,
                    SUM(ABS(il.quantity_delta)) AS total_units_sold,
                    SUM(ABS(il.quantity_delta)) / :days_lookback AS
avg_daily_sales
                FROM inventory_ledger il
                WHERE il.change_type = 'sale'
                  AND il.created_at >= :cutoff_date
                GROUP BY il.product_id, il.warehouse_id
            )
            SELECT
                p.id                  AS product_id,
                p.name                AS product_name,
                p.sku                 AS sku,
                w.id                  AS warehouse_id,
                w.name                AS warehouse_name,
                i.quantity            AS current_stock,
                i.reorder_point       AS threshold,
                COALESCE(rs.avg_daily_sales, 0) AS avg_daily_sales,
                -- days_until_stockout: NULL if no sales velocity (divide by zero
guard)
                CASE
                    WHEN COALESCE(rs.avg_daily_sales, 0) > 0
                    THEN FLOOR(i.quantity / rs.avg_daily_sales)
                    ELSE NULL
```

```
                END                   AS days_until_stockout,
                s.id                  AS supplier_id,
                s.name                AS supplier_name,
                s.contact_email        AS supplier_email
            FROM inventory i
            INNER JOIN products p ON p.id = i.product_id
            INNER JOIN warehouses w ON w.id = i.warehouse_id
            LEFT JOIN suppliers s ON s.id = p.supplier_id
            LEFT JOIN recent_sales rs
                ON rs.product_id = i.product_id
               AND rs.warehouse_id = i.warehouse_id
            WHERE
                w.company_id = :company_id         -- Scoped to company
                AND p.is_active = TRUE             -- Exclude archived products
                AND w.is_active = TRUE             -- Exclude inactive warehouses
                AND i.quantity < i.reorder_point   -- Only true low-stock items
                AND rs.sale_count IS NOT NULL      -- Must have recent sales
activity
                AND (:warehouse_id IS NULL OR w.id = :warehouse_id) -- Optional
filter
            ORDER BY
                days_until_stockout ASC NULLS LAST,  -- Most urgent first
                i.quantity ASC
        ''')

        results = db.session.execute(sql, {
            'company_id': company_id,
            'days_lookback': days_lookback,
            'cutoff_date': cutoff_date,
            'warehouse_id': warehouse_filter_id,
        }).fetchall()

        # —— Serialize results ——
        alerts = []
        for row in results:
            alert = {
                'product_id': row.product_id,
                'product_name': row.product_name,
                'sku': row.sku,
                'warehouse_id': row.warehouse_id,
                'warehouse_name': row.warehouse_name,
                'current_stock': row.current_stock,
                'threshold': row.threshold,
                'days_until_stockout': row.days_until_stockout,
                'supplier': None
            }
            # Attach supplier only if one is linked
            if row.supplier_id:
                alert['supplier'] = {
                    'id': row.supplier_id,
                    'name': row.supplier_name,
                    'contact_email': row.supplier_email,
                }
            alerts.append(alert)

        logger.info(
            f'Low-stock alerts fetched',
            extra={'company_id': company_id, 'alert_count': len(alerts),
'user_id': g.current_user.id}
        )
```

```
        return jsonify({
            'alerts': alerts,
            'total_alerts': len(alerts),
            'metadata': {
                'company_id': company_id,
                'lookback_days': days_lookback,
                'generated_at': datetime.utcnow().isoformat() + 'Z',
            }
        }), 200

    except Exception as e:
        logger.error('Error fetching low-stock alerts', exc_info=e)
        return jsonify({'error': 'Internal server error'}), 500
```

3.3 — Edge Cases Handled

Edge Case	How It's Handled	Code Location
Company not found / wrong company	403 returned if user.company_id != path company_id	Authorization check line 1
Product has no supplier	supplier field returns null (not an error)	Left JOIN + null check in serializer
Product with zero sales velocity	days_until_stockout returns null, not divide-by-zero	CASE WHEN avg_daily_sales > 0 in SQL
No alerts for this company	Returns {alerts: [], total_alerts: 0}	Empty list handled naturally
Invalid 'days' query param	Returns 400 with descriptive message	Param validation block
Product in multiple warehouses	Each (product, warehouse) pair is a separate alert	CTE groups by both product_id + warehouse_id
Bundle products alerting	Bundles included; NOTE component depletion separate	Assumption A7 documents this gap
Inactive warehouse/product	Excluded via is_active = TRUE WHERE clauses	SQL WHERE clause
Very high days_until_stockout	Sorted ascending, so urgent ones first; high values at end	ORDER BY days_until_stockout ASC NULLS LAST
Cross-company data access	company_id path param always scoped via WHERE w.company_id = :company_id	SQL + authorization check

3.4 — Sample API Response

```
// GET /api/companies/42/alerts/low-stock?days=30
// HTTP 200 OK
{
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 3,
      "supplier": {
        "id": 789,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com"
      }
    },
    {
      "product_id": 200,
      "product_name": "Bolt Pack XL",
      "sku": "BLT-200",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 8,
      "threshold": 25,
      "days_until_stockout": null,
      "supplier": null
    }
  ],
  "total_alerts": 2,
  "metadata": {
    "company_id": 42,
    "lookback_days": 30,
    "generated_at": "2026-04-29T10:00:00Z"
  }
}
```

3.5 — Performance & Scalability Considerations

Indexes Required for This Query

- inventory_ledger(change_type, created_at) — covers the CTE filter
- inventory_ledger(product_id, warehouse_id) — covers the CTE GROUP BY
- inventory(product_id, warehouse_id) — covers the main JOIN
- warehouses(company_id) — covers the WHERE company_id filter

For High-Traffic Companies

If a company has 10,000+ products across 50 warehouses, running this query on every request is expensive. Recommended approach:

- Materialize a low_stock_cache table refreshed every 5 minutes by a background job (Celery + Redis)
- The API reads from the cache table (O(1) SELECT) instead of running the full CTE
- Cache is invalidated on inventory_ledger insert — or just rebuilt on schedule

Rate Limiting

This endpoint should be rate-limited per company (e.g., 60 requests/minute) to prevent scraping or accidental polling loops from client integrations.

Summary & Submission Notes

This submission addresses all three parts of the StockFlow case study in full. Below is a quick reference summary:

Part	What Was Delivered	Key Highlights
Part 1: Debug	11 issues identified, explained, and fixed with full corrected code	Atomicity fix, auth, validation, error handling, logging
Part 2: Schema	9-table normalized SQL DDL + ER overview + 8 gap questions	Audit ledger, multi-tenant SKU, bundle support, soft deletes
Part 3: API	Full Flask endpoint with CTE SQL, auth, edge cases, sample response	Days-until-stockout, sales velocity, parameterized lookback

THANK YOU!